

实验二 树实验

基础练习一

【问题描述】

题目给出的 C 程序通过逐个插入字符建立一棵二叉排序树，并要求补全 `inorder` 函数中的三个空，再写出输入字符串 `ephqsbma` 时的中序遍历输出结果。为了验证填空后的逻辑，本实验额外用 Rust 重写了同样的建树与中序遍历过程。

程序输入为字符序列，逐个插入二叉排序树；输出为中序遍历得到的字符序列。对给定输入 `ephqsbma`，需要验证最终输出是否为按字典序排列的 `abehmpqs`。

【基本思路】

填空的关键在于理解中序遍历顺序为“左子树 -> 根结点 -> 右子树”。由于原程序的 `add` 按二叉排序树规则插入字符，较小字符进入左子树，较大或相等字符进入右子树，因此中序遍历结果会按非降序输出。

Rust 实现中用 `Option<Box<Node>>` 表示二叉链表，`add` 负责逐步插入字符，`inorder` 递归访问左子树、输出当前结点、再访问右子树。这样既能验证填空内容分别是 `lchild`、`data`、`rchild`，也能直接验证题目给定输入的运行结果。

<code>build_tree(text)</code>	<code>root <- 空树</code>	<code>for ch in text</code>	若 <code>ch</code> 不是空白字符，则 <code>add(root, ch)</code>
<code>inorder(node)</code>	递归访问左子树	输出 <code>node.data</code>	递归访问右子树

函数调用关系可表示为：`main()` -> 读取 `inputfile.txt` -> `build_tree()` -> `inorder_string()` -> 输出中序遍历结果。

【实验结果及分析】

1. 实验结果：

1.1 实验过程描述

- 1) 核心建树与遍历函数位于 `src/lib.rs`，主程序位于 `src/main.rs`，测试代码位于 `tests/test_fn.rs`；`ex_1/result.c` 中保留了填空后的 C 版本答案。
- 2) 测试数据覆盖题目给定输入、带空格输入、空输入和重复字符输入，重点验证插入规则与中序遍历次序。

编号	测试数据	预期结果 / 实际分析
1	输入 <code>ephqsbma</code>	中序遍历输出 <code>abehmpqs</code> ；与题目要求完全一致。
2	输入 <code>e p h q s b m a</code>	忽略空格后输出仍为 <code>abehmpqs</code> ；读取逻辑稳定。
3	空输入	返回空字符串；空树遍历不会出错。
4	输入 <code>abba</code>	输出 <code>aabb</code> ；重复字符按“相等放右子树”规则处理。

1.2 测试结果

运行 `cargo test` 后，本题 3 个单元测试全部通过；运行 `cargo run` 后，程序输出“中序遍历结果: abehmpqs”，与题目要求一致，因此原题空格应依次填写为 `lchild`、`data`、`rchild`，给定输入的输出为 `abehmpqs`。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于把二叉排序树插入规则与中序遍历顺序结合起来理解；难点较小，主要是要意识到原题虽然只要求填空，但要通过完整建树过程才能可靠验证最终输出。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是：既完成了原 C 程序填空，又用 Rust 做了自动化验证，避免只凭手算得出结果。可以改进的方向包括：直接把原 C 程序完整迁移为可编译版本，并补充对重复字符插入策略的说明。

3. 还有哪些实现方法？

还可以不显式建立整棵树，而是先手工分析插入后的树形结构再做中序遍历；或者使用数组统计字符后再排序输出，但那样就不能体现二叉排序树的建树过程。

基础练习二

【问题描述】

题目给出图 10-4 所示二叉树，要求建立其二叉链式存储结构，并实现结点数、叶子数、层次遍历、先序遍历、中序遍历、后序遍历、树深度以及指定结点为根的子树深度等基本操作。

本实验在 `ex_2` 中按图 10-4 直接构造二叉链表：根结点为 10，左子树包含 3、2、4、9、8、9，右子树包含 18、13、21。主程序演示各项操作，测试代码验证统计结果与遍历序列。

【基本思路】

存储结构上使用 `Option<Box<Node>>` 表示二叉链表，`BinaryTree` 对外提供各项操作。先序、中序、后序遍历都使用递归实现；层次遍历使用队列按层访问；树深度使用“左右子树最大深度 + 1”的递归公式计算。

对子树深度的求法是：先在树中查找值为 `x` 的结点，找到后再对该结点作为根调用深度计算函数。本实验图中数值 9 出现两次，因此代码对“按值查找”采用先序遇到的第一个匹配结点；演示时使用结点 4 和 18 避免歧义。

<code>count_nodes(node) / count_leaves(node)</code>	递归统计左右子树	<code>preorder / inorder / postorder</code>
按对应次序递归访问	<code>level_order(root)</code>	根结点入队，循环出队并把左右孩子入队
<code>depth(node)</code>	<code>max(depth(left), depth(right)) + 1</code>	

函数调用关系可表示为：main() -> BinaryTree::sample_tree() -> 统计函数 / 遍历函数 / depth() / subtree_depth() -> 输出结果。

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

- 3) 本部分代码位于 ex_2/src/lib.rs，主程序位于 ex_2/src/main.rs，测试代码位于 ex_2/tests/test_fn.rs。程序直接构造题图中的二叉树，然后输出结点数、叶子数、层次遍历、三种深度优先遍历结果以及树深度。
- 4) 测试数据围绕题目要求的七类操作展开，重点验证递归遍历次序是否正确、队列层次遍历是否按层输出，以及指定结点为根时的子树深度是否正确。

编号	测试数据	预期结果 / 实际分析
1	图 10-4 整棵二叉树	结点数=10，叶子数=5；说明树结构建立正确。
2	对整棵树做层次、先序、中序、后序遍历	层次遍历为 [10,3,18,2,4,13,21,9,8,9]，其余三种遍历结果也与程序输出一致。
3	查询树深度、以 4 为根的子树深度、以 18 为根的子树深度	分别得到 5、3、2；说明递归深度计算正确。
4	查询不存在的结点 x=100	返回 None；说明异常输入能被识别。

1.2 测试结果

运行 cargo test 后，本题 3 个单元测试全部通过；运行 cargo run 后，程序输出结点数 10、叶子数 5、层次遍历 [10, 3, 18, 2, 4, 13, 21, 9, 8, 9]、树深度 5，以及以 4 为根的子树深度 3，均与预期一致。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于把一棵给定图形的树准确映射为二叉链表，并分别实现递归遍历、层次遍历和深度计算；难点在于不同操作共享同一棵树结构，任何一个链接关系写错都会导致多项结果同时出错。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是：树结构定义统一，遍历和统计函数职责清晰，测试可以同时覆盖多个操作结果。可以改进的方向包括：把树结构改为从输入文件读取；对于子树查询增加按结点编号而不是按键值查找，以避免重复值带来的歧义。

4. 还有哪些实现方法？

还可以使用数组顺序存储这棵树，再由下标关系实现部分操作；也可以用显式栈改写三种深度优先遍历，减少递归写法带来的调用层次。

进阶练习一

【问题描述】

题目给出图 10-5 所示二叉树及其顺序表 sa, 其中 sa.elem[1..last] 为 A B C D ⊙ E F ⊙ G ⊙ ⊙ H ⊙, 要求由该顺序存储结构建立对应的二叉链表。

本实验在 ex_3 中把顺序表中的有效结点记为 Some(char), 空位置记为 None, 然后根据完全二叉树下标关系建立二叉链表。建树完成后, 程序输出层次遍历、先序遍历、中序遍历、后序遍历和树深度, 用来验证生成结果是否与图 10-5 一致。

【基本思路】

核心思路是递归处理顺序表下标 index: 若当前位置为空或越界, 则返回空指针; 否则创建当前结点, 并递归建立其左孩子 index*2 与右孩子 index*2+1。这样可以直接把顺序存储中的位置关系翻译成二叉链式结构。

由于顺序表中含有多个 ⊙ 空位, 建树时必须保留这些空位对后续下标的影响。例如 G 位于下标 9, H 位于下标 12, 如果忽略前面的空位, 生成出的链表结构就会与原图不一致。

```
build_node(data, index)  若 index 越界或 data[index] 为空, 则返回空  创建当前结点
left  <-  build_node(data,  index*2)          right  <-  build_node(data,  index*2+1)
BinaryTree::from_seq_table(data)  root <- build_node(data, 1)
```

函数调用关系可表示为: main() -> sample_seq_table() -> BinaryTree::from_seq_table() -> 各种遍历函数 / depth() -> 输出结果。

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

- 5) 本部分代码位于 ex_3/src/lib.rs, 主程序位于 ex_3/src/main.rs, 测试代码位于 ex_3/tests/test_fn.rs。程序把题目顺序表写成 Option<char> 数组, 并据此自动建立二叉链表。
- 6) 测试重点是检查空位是否被正确保留, 以及由此得到的层次遍历、先序遍历、中序遍历、后序遍历和树深度是否与图 10-5 对应。

编号	测试数据	预期结果 / 实际分析
1	顺序表 [A,B,C,D,⊙,E,F,⊙,G,⊙,⊙,H,⊙]	可成功建立二叉链表, 层次遍历得到 [A,B,C,D,E,F,G,H]。
2	对建立后的树做先序遍历	结果为 [A,B,D,G,C,E,H,F]; 与图 10-5 的根左右访问顺序一致。
3	对建立后的树做中序、后序遍历	中序为 [D,G,B,A,H,E,C,F], 后序为 [G,D,B,H,E,F,C,A]; 说明左右孩子连接关系正确。
4	检查空位保留与树深度	⊙ 空位未被错误补齐, 树深度为 4; 说明按下标递归建树正确。

1.2 测试结果

运行 cargo test 后，本题 3 个单元测试全部通过；运行 cargo run 后，程序输出层次遍历 ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']、先序遍历 ['A', 'B', 'D', 'G', 'C', 'E', 'H', 'F']、中序遍历 ['D', 'G', 'B', 'A', 'H', 'E', 'C', 'F']、后序遍历 ['G', 'D', 'B', 'H', 'E', 'F', 'C', 'A']，树深度为 4。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于理解顺序存储下标与二叉链表父子关系的对应规则；难点在于不能简单跳过空位，否则后续结点的左右关系会整体错位。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是：建树函数短而直接，完全按照 index 、 $2*\text{index}$ 、 $2*\text{index}+1$ 的关系递归展开，便于和教材中的顺序存储定义对应。可以改进的方向包括：把顺序表改为从文本文件解析；或者在建树后补充图形化输出帮助人工核对。

5. 还有哪些实现方法？

还可以先扫描顺序表创建结点数组，再第二轮统一连接左右孩子；也可以使用队列按层建立二叉链表，但递归按下标建树更贴近本题定义。

附加题

【问题描述】

附加要求是在树的孩子表示法基础上实现“求孩子”算法。本实验在 ex_4 中构造一个简单的树：A 的孩子为 B、C、D，B 的孩子为 E、F，D 的孩子为 G，并实现按双亲结点值返回其全部孩子结点序列。

实现目标是：在孩子表示法中先找到双亲结点所在位置，再顺着该结点的孩子链表依次取出所有孩子。如果双亲不存在，返回 None；如果双亲是叶子结点，则返回空数组。

【基本思路】

存储结构上采用“结点顺序表 + 每个结点对应一条孩子单链表”的方式。TreeNode 保存结点值和 first_child 指针，ChildNode 保存某个孩子在结点表中的下标以及下一兄弟指针。这样既保留了孩子表示法的结构，也便于顺序访问全部孩子。

求孩子算法分两步：第一步顺序查找双亲结点下标；第二步从 first_child 出发，沿 next 指针逐个访问兄弟结点，把它们对应的数据域追加到结果数组中。整个过程直观，适合作为树结构入门实现。

```
child_values(parent)  找到 parent 对应下标 parent_index  result <- 空数组  current <-
nodes[parent_index].first_child  while current 非空  取出 child_index 对应的数据加入
result  current <- current.next
```

函数调用关系可表示为：main() -> sample_tree() -> child_values(parent) -> 输出某个双亲结点的全部孩子。

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

- 7) 本部分代码位于 `ex_4/src/lib.rs`, 主程序位于 `ex_4/src/main.rs`, 测试代码位于 `ex_4/tests/test_fn.rs`。程序分别演示 A、B、D、G 四个结点的求孩子结果, 其中 G 为叶子结点, 用来验证空孩子表处理。
- 8) 测试数据覆盖多孩子结点、单孩子结点、叶子结点和不存在的双亲结点, 重点验证孩子链表的遍历顺序和异常处理。

编号	测试数据	预期结果 / 实际分析
1	求 A 的孩子结点	返回 [B,C,D]; 验证多孩子结点的遍历顺序。
2	求 B 的孩子结点	返回 [E,F]; 验证链表中多个兄弟结点访问正确。
3	求 D 的孩子结点	返回 [G]; 验证单孩子结点情况。
4	求 G 的孩子结点 / 求不存在的结点 Z	G 返回 [], Z 返回 None; 说明叶子结点与非法双亲都能正确处理。

1.2 测试结果

运行 `cargo test` 后, 本题 3 个单元测试全部通过; 运行 `cargo run` 后, 程序输出 A 的孩子结点 ['B', 'C', 'D'], B 的孩子结点 ['E', 'F'], D 的孩子结点 ['G'], G 的孩子结点 [], 与设计一致。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于理解孩子表示法中“结点表 + 孩子链表”的两层结构; 难点在于既要找到双亲, 又要正确维护孩子链表中各兄弟结点的先后次序。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是: 结构简单, 能够清楚展示求孩子算法的访问过程, 且对不存在双亲与叶子结点都做了区分处理。可以改进的方向包括: 增加删除孩子、插入兄弟等操作; 或者把字符结点扩展为泛型数据。

6. 还有哪些实现方法?

还可以使用双亲表示法后通过整表扫描反查所有孩子; 也可以把树改写为“长子-兄弟”表示法, 再通过 `first_child` 与 `next_sibling` 实现类似查询。